KHAN INSTITUTE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

COMPUTER SCIENCE DEPARTMENT

Assignment #01 – Theory Report

AI-Based Application Development in .NET

AI General Knowledge Q&A Chatbot in C#

Submitted by:

Aleeza Maryam

Roll No: 232201025

Submitted to:

Sir Uzair Hassan

Department of Computer Science

March 29, 2026

Contents

1 Introduction to AI-Based Applications **3**

1.1 What is Artificial Intelligence? 3

1.2 What are AI-Based Applications? 3

1.3 Real-World Examples 4

2 AI in the .NET Ecosystem **5**

2.1 How AI is Integrated in .NET 5

2.2 Introduction to ML.NET 5

2.3 REST APIs for AI Services 5

2.4 Benefits of Using .NET for AI Applications 6

3 AI Application Development Lifecycle **7**

3.1 1. Problem Identification 7

3.2 2. Data Collection 7

3.3 3. Data Preprocessing 7

3.4 4. Model Selection 7

3.5 5. Model Training 8

3.6 6. Model Evaluation 8

3.7 7. Integration with Application 8

3.8 8. Deployment 8

4 Architecture of an AI-Based Application **9**

4.1 System Architecture 9

4.2 Frontend (UI – Windows Forms) 9

4.3 Backend (.NET Logic) 9

4.4 AI Model / API 10

5 API Integration Concept **11**

5.1 What is an API? 11

5.2 How AI APIs are Used in Applications 11

5.3 Request/Response Flow 11

6 Introduction to Docker **12**
6.1 What is Docker? 12
6.2 Why Docker is Used 12
6.3 Benefits of Docker 12
6.4 Dockerfile for This Project 12

7 Version Control using GitHub **14**
7.1 Importance of GitHub 14
7.2 Basic GitHub Workflow 14
7.2.1 Repository Creation 14
7.2.2 Commit and Push 14
7.2.3 README File 15

8 Ethical Considerations **16**
8.1 Bias in AI 16
8.2 Data Privacy 16
8.3 Responsible AI Usage 16

Conclusion **17**

1. Introduction to AI-Based Applications

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think, learn, and make decisions. AI enables computers to perform tasks that typically require human intelligence, such as understanding natural language, recognizing patterns, solving problems, and making decisions.

AI is broadly categorized into:

- **Narrow AI (Weak AI):** Designed for a specific task, such as voice assistants or recommendation systems.
- **General AI (Strong AI):** Hypothetical AI that can perform any intellectual task a human can.
- **Super AI:** A future concept where AI surpasses human intelligence in all areas.

1.2 What are AI-Based Applications?

AI-based applications are software systems that use artificial intelligence techniques to provide intelligent functionality. These applications can learn from data, adapt to new inputs, and perform tasks autonomously.

Key characteristics of AI-based applications include:

- **Learning:** Ability to improve performance over time using data.
- **Reasoning:** Ability to solve problems and make logical decisions.
- **Self-correction:** Ability to learn from mistakes and improve accuracy.
- **Natural Language Processing:** Ability to understand and generate human language.

1.3 Real-World Examples

Application	Description
Chatbots	AI-powered conversational agents like ChatGPT and Google Gemini that answer questions in natural language.
Recommendation Systems	Netflix, YouTube, and Amazon use AI to suggest content or products based on user behavior.
Image Recognition	Google Photos and Face ID use AI to identify faces, objects, and scenes in images.
Virtual Assistants	Siri, Alexa, and Google Assistant use AI for voice recognition and task automation.
Medical Diagnosis	AI systems assist doctors in detecting diseases from X-rays and MRI scans.

2. AI in the .NET Ecosystem

2.1 How AI is Integrated in .NET

Microsoft's .NET platform provides powerful tools and libraries for integrating AI into applications. Developers can build AI-powered applications using two main approaches:

1. **ML.NET:** A machine learning framework built specifically for .NET developers.
2. **REST API Integration:** Connecting to external AI services such as Google Gemini or OpenAI through HTTP requests.

2.2 Introduction to ML.NET

ML.NET is an open-source, cross-platform machine learning framework developed by Microsoft. It allows .NET developers to build, train, and deploy machine learning models without requiring deep knowledge of data science.

Key features of ML.NET:

- Supports classification, regression, clustering, and anomaly detection.
- Integrates with Visual Studio through Model Builder (GUI tool).
- Can import pre-trained TensorFlow and ONNX models.
- Works with C# and F# languages.

2.3 REST APIs for AI Services

REST (Representational State Transfer) APIs allow applications to communicate with external AI services over the internet. In this project, we use the **Google Gemini API** to provide AI-powered question answering.

The process involves:

1. Sending an HTTP POST request with the user's question.
2. The AI service processes the request and generates a response.

3. The application receives and displays the response.

2.4 Benefits of Using .NET for AI Applications

- **Cross-platform:** .NET 6+ runs on Windows, Linux, and macOS.
- **Performance:** .NET is known for high performance and scalability.
- **Rich ecosystem:** Large library support including HttpClient, System.Text.Json.
- **Easy API integration:** Built-in HTTP support makes REST API calls straightforward.
- **Strong typing:** C# provides compile-time error checking for safer code.

3. AI Application Development Lifecycle

The development of an AI-based application follows a structured lifecycle to ensure quality and effectiveness.

3.1 1. Problem Identification

The first step is clearly defining the problem the AI will solve. In this project, the problem is: *"Users need a conversational interface to get instant answers to general knowledge questions without searching through multiple web pages."*

3.2 2. Data Collection

For API-based AI applications like this project, data collection involves gathering sample questions and expected answers to test the system. For ML.NET-based applications, large datasets are collected from various sources.

3.3 3. Data Preprocessing

Raw data is cleaned, formatted, and prepared for use. In our project, user input is trimmed, validated, and formatted into JSON before being sent to the API.

3.4 4. Model Selection

The appropriate AI model is selected based on the task. For this project, **Google Gemini 1.5 Flash** was selected because:

- It is optimized for fast responses.
- It supports conversational context (chat history).
- It is available on a free tier for development.

3.5 5. Model Training

Since we are using a pre-trained API model, this step is handled by Google. For ML.NET projects, training involves feeding data to the model and adjusting parameters.

3.6 6. Model Evaluation

The model is tested with various sample questions to evaluate:

- Accuracy of responses.
- Response time.
- Handling of edge cases (empty input, invalid questions).

3.7 7. Integration with Application

The AI model is integrated into the Windows Forms application using `HttpClient` to make API calls and `System.Text.Json` to parse responses.

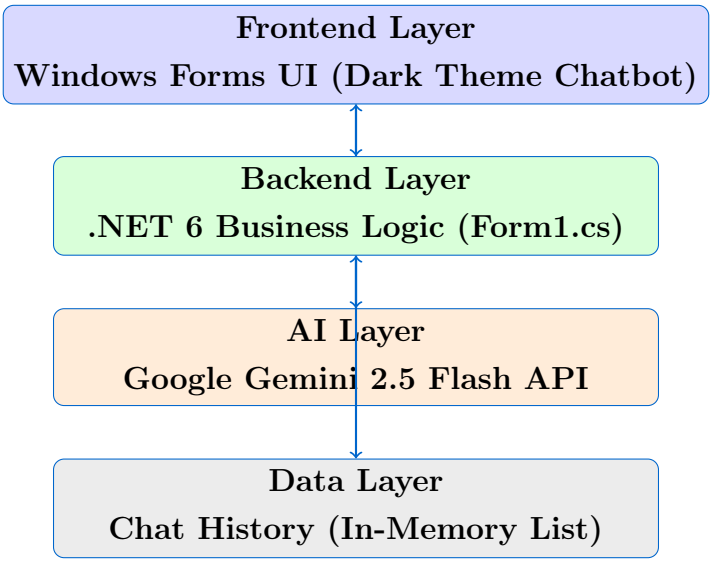
3.8 8. Deployment

The application is packaged using Docker for consistent deployment across different environments, and the source code is hosted on GitHub for version control.

4. Architecture of an AI-Based Application

4.1 System Architecture

The architecture of this AI Q&A application consists of three main layers:



4.2 Frontend (UI – Windows Forms)

The user interface is built using Windows Forms with a dark theme. It includes:

- A chat display area (RichTextBox) showing messages as bubbles.
- A text input field for typing questions.
- Send and Clear buttons.
- Status bar showing AI response state.

4.3 Backend (.NET Logic)

The application logic handles:

- Input validation and error handling.

- Building and serializing JSON request bodies.
- Making asynchronous HTTP calls using `HttpClient`.
- Parsing API responses using `System.Text.Json`.
- Maintaining conversation history in a `List<object>`.

4.4 AI Model / API

Google Gemini 1.5 Flash API processes the questions and generates intelligent responses. The API receives the full conversation history to maintain context across multiple questions.

5. API Integration Concept

5.1 What is an API?

An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate with each other. APIs act as intermediaries that enable applications to access features or data from external services.

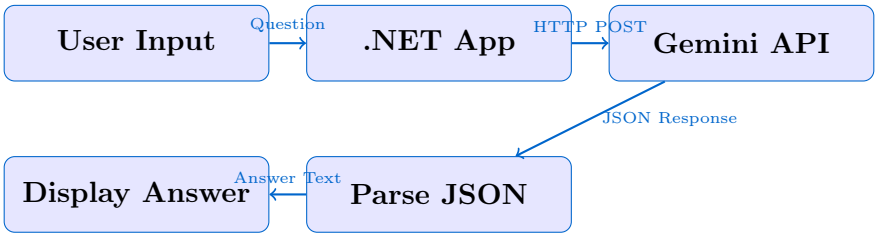
5.2 How AI APIs are Used in Applications

AI APIs provide pre-trained intelligence as a service. Instead of building and training models from scratch, developers send requests to AI APIs and receive intelligent responses.

In this project:

1. The user types a question in the Windows Forms UI.
2. The application sends an HTTP POST request to the Google Gemini API.
3. The API processes the question using its AI model.
4. The API returns a JSON response containing the answer.
5. The application parses the JSON and displays the answer.

5.3 Request/Response Flow



6. Introduction to Docker

6.1 What is Docker?

Docker is an open-source platform that enables developers to package applications and their dependencies into lightweight, portable containers. A container includes everything needed to run the application: code, runtime, libraries, and configuration files.

6.2 Why Docker is Used

Without Docker, an application may work on a developer's machine but fail on another computer due to differences in environment, installed software, or configurations. Docker solves this problem by ensuring the application runs the same way everywhere.

6.3 Benefits of Docker

Benefit	Description
Portability	Run the same container on any machine — Windows, Linux, or cloud.
Consistency	Eliminates the "it works on my machine" problem.
Isolation	Each container runs independently without affecting other applications.
Scalability	Easily scale applications by running multiple containers.
Fast Deployment	Containers start in seconds compared to virtual machines.

6.4 Dockerfile for This Project

```
FROM mcr.microsoft.com/dotnet/sdk:6.0 AS build
WORKDIR /app
```

```
COPY AIQnA.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/runtime:6.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
CMD ["dotnet", "AIQnA.dll"]
```

7. Version Control using GitHub

7.1 Importance of GitHub

GitHub is a cloud-based platform for version control and collaboration. It allows developers to:

- Track changes in source code over time.
- Collaborate with team members on the same project.
- Maintain a backup of the entire project history.
- Share code publicly or privately.

7.2 Basic GitHub Workflow

7.2.1 Repository Creation

A repository (repo) is a storage location for the project. For this project, a public repository named `AIQnA` was created on GitHub containing all source files.

7.2.2 Commit and Push

1. **Commit:** Saving a snapshot of changes with a descriptive message.
2. **Push:** Uploading committed changes to the GitHub repository.

Files uploaded to the repository:

- `Form1.cs` — Main application logic
- `Form1.Designer.cs` — UI layout
- `Program.cs` — Entry point
- `AIQnA.csproj` — Project configuration
- `Dockerfile` — Container setup
- `README.md` — Project description

- `.gitignore` — Excludes sensitive files

7.2.3 README File

The README.md file provides:

- Project description and features.
- Setup and installation instructions.
- Screenshots of the application.
- Docker build and run commands.

8. Ethical Considerations

8.1 Bias in AI

AI models are trained on large datasets that may contain human biases. These biases can lead to unfair or inaccurate responses. For example, an AI trained on biased data may give different quality answers based on the cultural context of the question.

Mitigation:

- Use diverse and representative training data.
- Regularly audit AI outputs for bias.
- Allow users to report incorrect or biased responses.

8.2 Data Privacy

AI applications often process sensitive user data. It is important to:

- Never store user questions without consent.
- Keep API keys secure and never expose them publicly.
- Follow data protection regulations such as GDPR.

In this project, the API key is stored locally and excluded from GitHub using `.gitignore` to prevent unauthorized access.

8.3 Responsible AI Usage

Developers must ensure AI is used ethically and responsibly:

- **Transparency:** Clearly inform users they are interacting with AI.
- **Accountability:** Take responsibility for the AI's outputs.
- **Fairness:** Ensure the AI treats all users equally.
- **Safety:** Prevent the AI from generating harmful content.

Conclusion

This report has covered the complete theoretical foundation of AI-based application development using the .NET ecosystem. The key topics explored include the nature of artificial intelligence, its integration in .NET through ML.NET and REST APIs, the full development lifecycle, system architecture, Docker containerization, GitHub version control, and ethical considerations.

The practical implementation — an AI General Knowledge Q&A Chatbot built with Windows Forms and powered by Google Gemini API — demonstrates how these theoretical concepts translate into a real-world working application.